CS 450 · FINAL PROJECT

Credit Card Fraud Detection

Building and comparing supervised learning models to identify fraudulent transactions

Group 10

Parleen Bagga
Adarsh Shresth
Khush Patel
Marco Novellini Blasco

The Problem and Why It Matters

$33B+

Lost annually to credit card fraud worldwide

0.087

Fraction of transactions in our data that are fraud

Real-time

Banks must decide in milliseconds, at scale

PROJECT GOAL

Build a supervised learning system that classifies credit card transactions as fraud or legitimate, evaluate it with metrics that handle severe class imbalance, and analyze which transaction features matter most.

WHY IT'S HARD

- Severe class imbalance — naive 'always legit' guesser scores 91% accuracy yet catches zero fraud
- Asymmetric costs — missing a fraud (false negative) is far more expensive than a false alarm
- Interpretability matters — banks must justify every blocked transaction to the customer

Dataset and Exploratory Analysis

DATASET AT A GLANCE

1,000,000 credit card transactions (Kaggle)

7 features + 1 binary label (fraud)

8.74% fraud rate — severe imbalance

0 missing values, all numeric

200K stratified sample for fast iteration

FEATURES

- distance_from_home
- distance_from_last_transaction
- ratio_to_median_purchase_price
- repeat_retailer, used_chip, used_pin_number, online_order (binary)

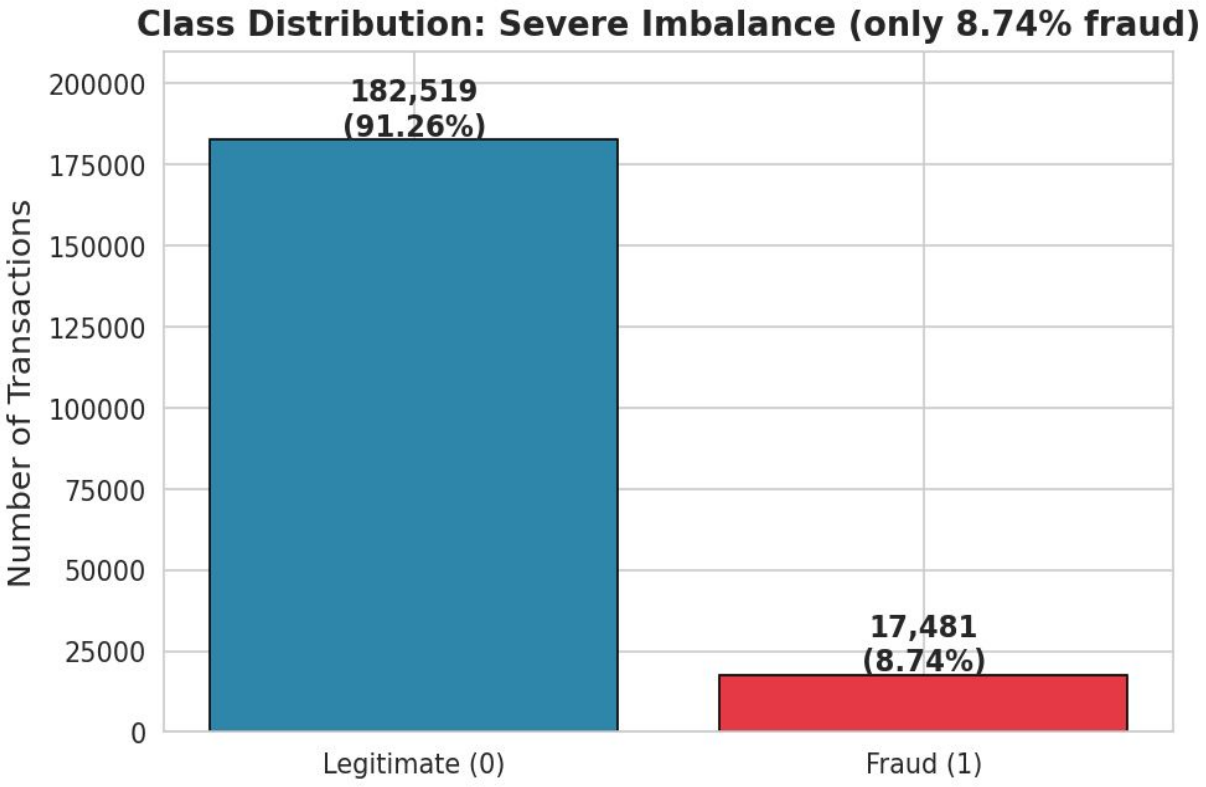


Fig 1. The 91/9 split: why accuracy is the wrong metric for fraud

Key insight: ratio_to_median_purchase_price has the strongest correlation with fraud (r = 0.46) — fraudsters tend to make purchases far larger than your usual spending.

Pipeline and Models

01	02	03	04	05
Load & EDA	**Stratified Split**	**Scale (LR only)**	**Train 3 Models**	**Evaluate**
Inspect class balance, distributions, correlations	80/20 train/test, preserves 8.74% fraud ratio	StandardScaler — fit on train, transform test	All with class_weight='balanced'	Precision, Recall, F1, ROC-AUC, PR-AUC

THREE SUPERVISED LEARNING MODELS

Logistic Regression

Linear baseline

Models log-odds of fraud as a weighted sum of features. Fast, interpretable coefficients. Sensitive to feature scale — hence the StandardScaler step.

Decision Tree

Single non-linear learner

Recursively splits the data on the most informative feature. Captures interactions that LR cannot, but a single tree easily overfits.

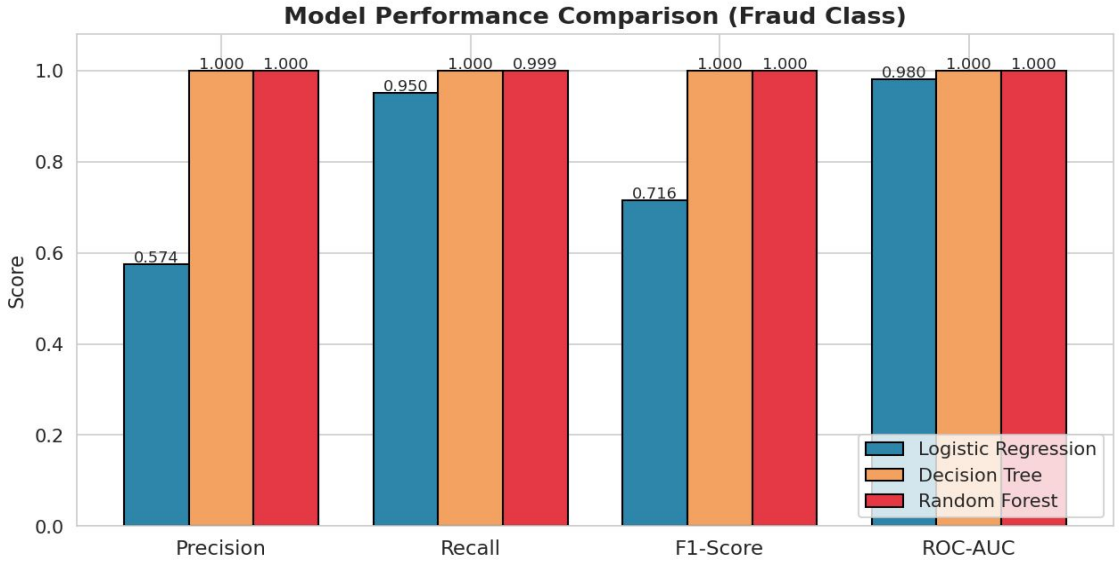
Random Forest

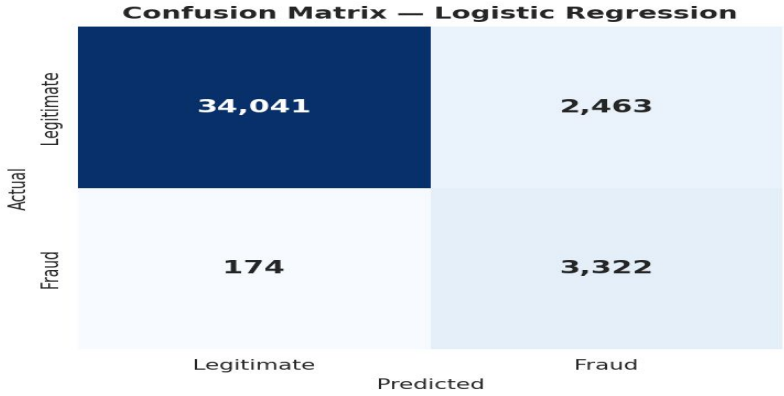
Ensemble of 100 trees

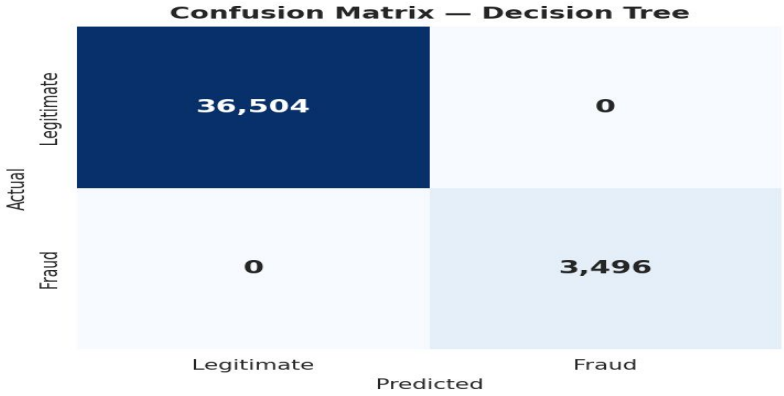
Each tree sees a bootstrap sample and a random feature subset at each split. Majority vote reduces variance — usually the strongest performer.

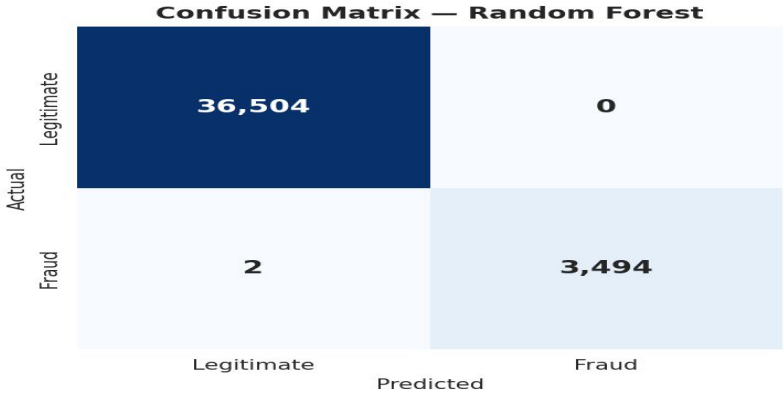
Model Comparison: Where the Numbers Land

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.574	0.950	0.716	0.9802
Decision Tree	1.000	1.000	1.000	1.0000
Random Forest	**1.000**	**0.999**	**1.000**	**1.0000**

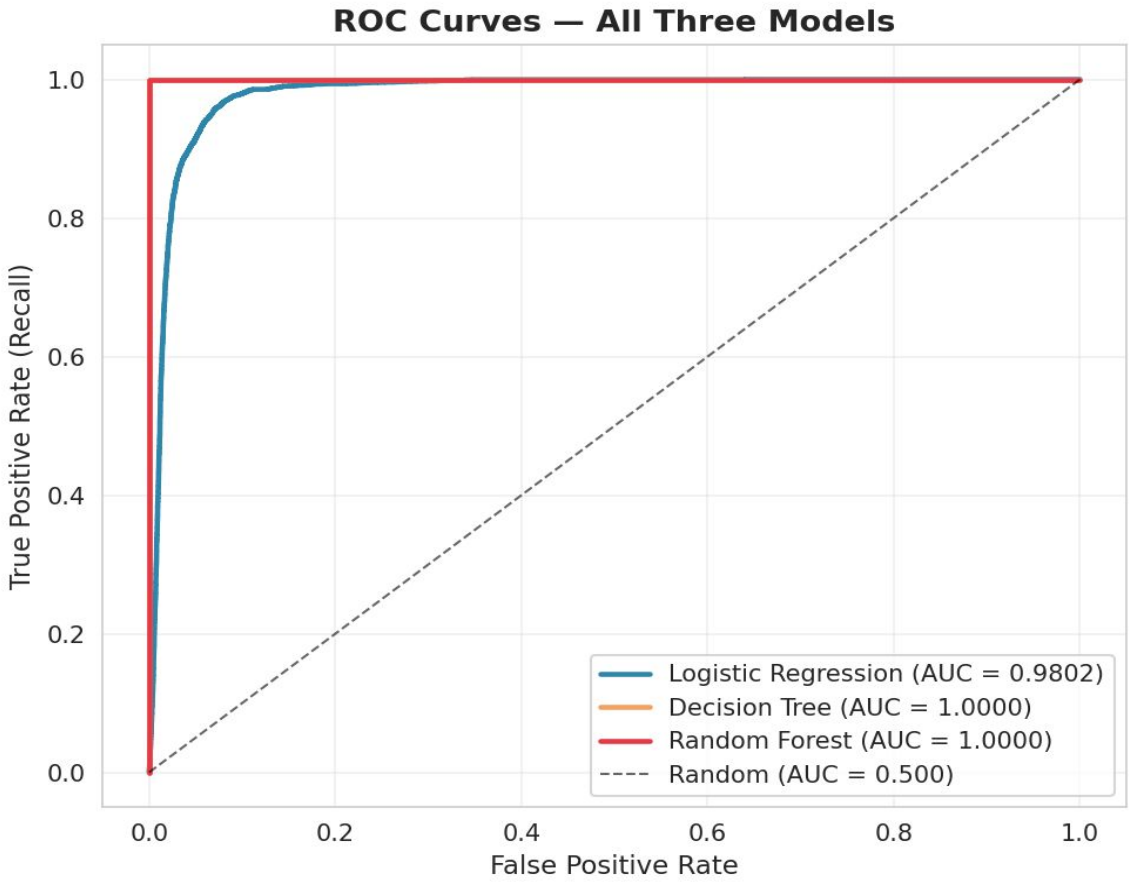


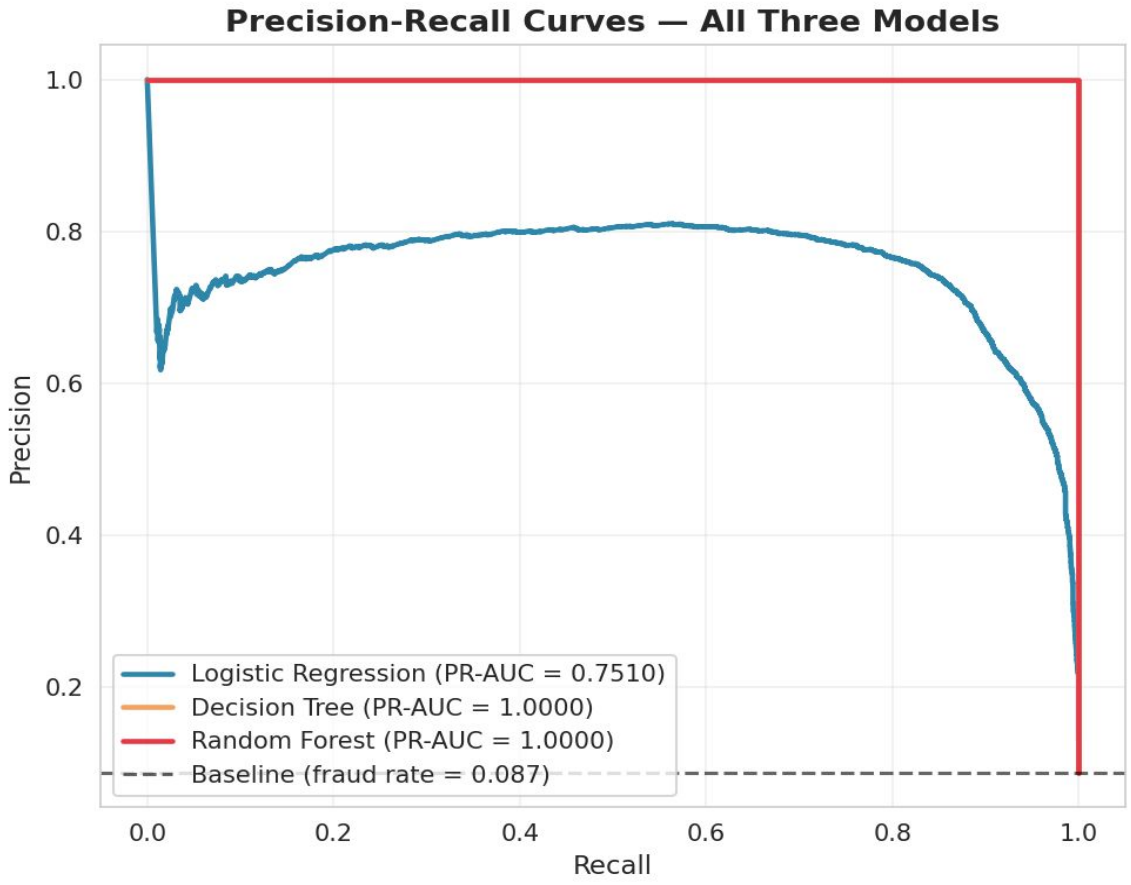






ROC and Precision-Recall Curves

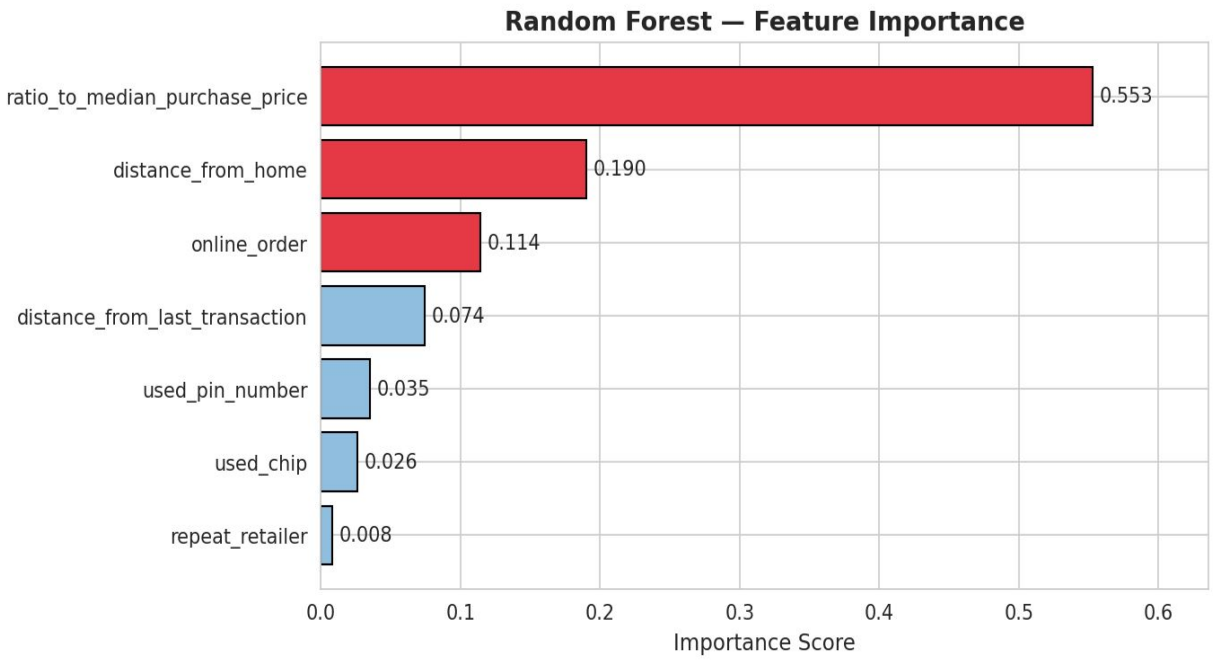




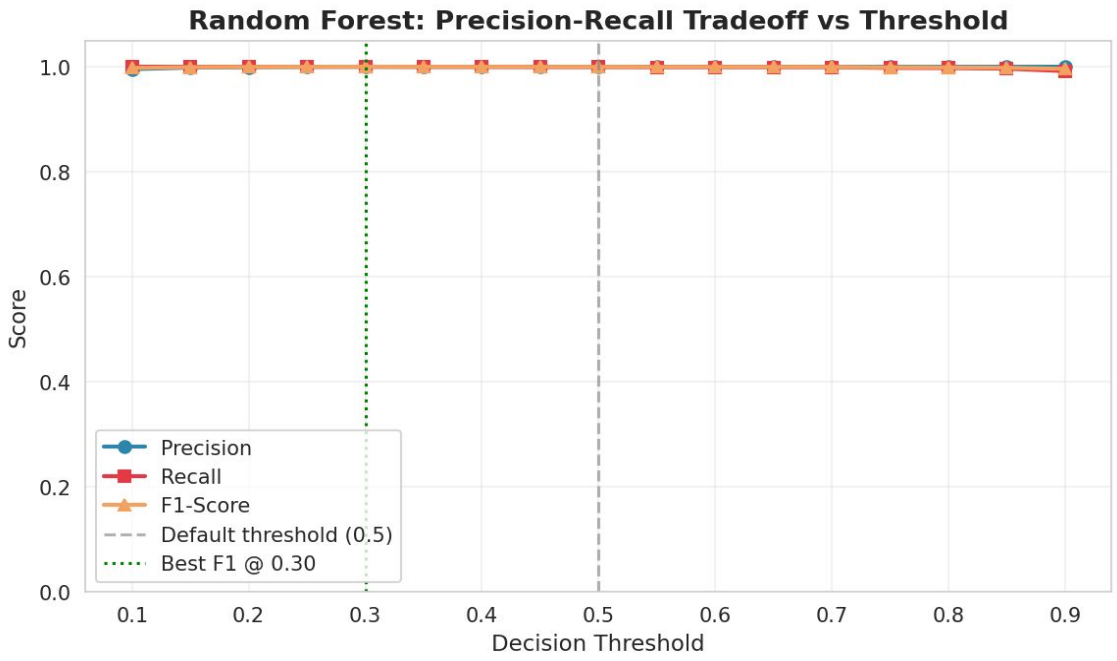
READING THE CURVES: Tree models hug the top-left corner (perfect separation). Logistic Regression is excellent (AUC 0.98) but trades precision for recall — it catches almost all fraud while flagging more legitimate transactions.

What the Model Learned

FEATURE IMPORTANCE (Random Forest)



THE PRECISION-RECALL TRADEOFF



Top 3 signals: ratio_to_median_purchase_price (55%), distance_from_home (19%), online_order (11%) — together they explain 85% of the model's decisions.

Business lever: lowering the decision threshold catches more fraud (higher recall) but raises false alarms. Banks tune this based on cost — a missed $5,000 fraud usually outweighs a brief customer inconvenience.

Honest Critique and Future Work

STRENGTHS

- Class imbalance handled correctly via stratified split + class_weight
- Three models compared on appropriate metrics (not accuracy)
- Feature importance gives interpretable, actionable insights
- Threshold tuning shows we understand the business side, not just the math

CHALLENGES WE HIT

- Choosing the right metrics — accuracy is misleading on 91/9 data
- Random Forest training time on 1M rows — solved with stratified sampling
- Avoiding data leakage — fit scaler on training set only

HONEST LIMITATIONS

- Synthetic dataset — fraud patterns are cleanly separable; tree models hit ~100% which is unrealistic
- On real data (e.g. Kaggle European set), state-of-the-art PR-AUC is ~0.85, not 1.00
- No temporal structure — real fraud evolves; we trained on a static snapshot

FUTURE DIRECTIONS

- Test on real-world PCA-anonymized data (Kaggle European set)
- Try gradient boosting (XGBoost, LightGBM) — typical winner on tabular fraud
- Apply SMOTE oversampling and compare against class_weight approach
- Deploy as a real-time API with sub-100ms inference

Conclusion

What we built, what we learned, why it matters

KEY FINDINGS

- Random Forest matched Decision Tree on every metric and is far more robust
- ratio_to_median_purchase_price drives 55% of model decisions
- Class weighting + stratification beat naive accuracy-driven training

LESSONS LEARNED

- Pick metrics that match the problem — accuracy hides the truth on imbalanced data
- Always question results that look too good — investigate, don't celebrate
- Interpretability is a feature, not a luxury, in regulated domains

REAL-WORLD IMPACT

- The same pipeline scales to insurance, healthcare, and cybersecurity anomaly detection
- Threshold tuning lets product teams trade off customer experience against risk
- Feature importance tells fraud analysts where to focus investigations

Thank you · Questions?

APPENDIX

Key Code Snippets

Github - Link

01 · Stratified Train/Test Split

Preserves the 8.74% fraud ratio identically in both train and test sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,        # <-- key: keeps fraud ratio
    random_state=42
)
```

02 · Handling Imbalance at Training

Penalizes fraud misses more heavily without generating synthetic data

```
rf = RandomForestClassifier(
    n_estimators=100,
    class_weight='balanced', # <-- key: upweights fraud
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train, y_train)
```

03 · Custom Decision Threshold

Default 0.5 cutoff isn't optimal — tune to balance precision vs recall

```
# Default: predict fraud if probability >= 0.5
y_pred_default = model.predict(X_test)

# Custom threshold: catch more fraud, accept more false alarms
threshold = 0.30
y_pred_custom = (
    model.predict_proba(X_test)[:, 1] >= threshold
).astype(int)
```

04 · Why We Use ROC-AUC + PR-AUC

Accuracy is misleading on 91/9 data — these metrics actually measure fraud detection

```
from sklearn.metrics import (
    roc_auc_score,
    average_precision_score   # PR-AUC
)
roc   = roc_auc_score(y_test, y_proba)
prauc = average_precision_score(y_test, y_proba)

# Naive all-legit model scores:
#   accuracy = 91%    ← misleading
#   PR-AUC   = 0.087  ← exposes it
```